SecureVault Project Workflow

SecureVault

A Spring Boot + React password vault with JWT authentication, encrypted credential storage, email verification, password reset, and sharing between registered users.

1. Project Purpose

SecureVault is a credential manager. A user registers, verifies email, logs in, stores passwords, updates them later, and shares selected entries with another registered user. Passwords are stored encrypted in PostgreSQL and decrypted only when returned through authorized API calls.

2. Tech Stack

Backend

- Spring Boot 4.1
- Spring Web MVC
- Spring Security
- Spring Data JPA
- PostgreSQL
- JJWT for JWT tokens
- Java Mail Sender

Frontend

- React
- Axios
- React Router
- Dashboard-based credential UI

3. High-Level Architecture

- `frontend` sends REST requests to `backend`.
- `AuthController` handles registration, login, email verification, refresh token, logout, and password reset.
- `PasswordVaultController` handles vault CRUD and sharing.
- `JwtAuthenticationFilter` reads the access token and sets the authenticated user in the security context.
- `AuthServiceImpl` manages account lifecycle and tokens.
- `PasswordVaultServiceImpl` manages encrypted passwords and sharing logic.
- PostgreSQL stores users, password entries, refresh tokens, reset tokens, and share records.

4. Main User Flows

1. **Register**: user submits name, email, username, and password. Backend validates the password, checks uniqueness, saves the user with status `PENDING`, and sends a verification email.
2. **Verify Email**: user clicks the email link. Backend marks the account `ACTIVE` and `emailVerified=true`.
3. **Login**: user enters email or username plus password. Backend authenticates credentials and returns access token + refresh token.
4. **Load Dashboard**: frontend calls `/api/auth/me`, `/api/vault`, and `/api/vault/shared`. It loads own entries and shared entries.
5. **Create Password**: user saves a credential. Backend encrypts the password and stores it in `password_entries`.
6. **Edit Password**: user updates an entry. Backend re-encrypts the password and saves the new row state.
7. **Share Password**: user selects an entry and a recipient email. Backend creates a row in `shared_password_entries` linking owner, recipient, permission, and share state.
8. **View Shared Password**: recipient opens the Shared Vault tab. Backend returns shared entries and the decrypted password so the recipient can view or copy it.
9. **Logout**: frontend clears the access token and backend removes refresh token state.

5. Authentication Workflow

Step	What happens
Register	`POST /api/auth/register` creates the user and sends verification email.
Verify email	`GET /api/auth/verify` activates the account.
Login	`POST /api/auth/login` returns access token + refresh token.
Refresh token	`POST /api/auth/refresh-token` issues a new access token if the refresh cookie is valid.
Logout	`POST /api/auth/logout` deletes the refresh token.

6. Password Vault Workflow

Endpoint	Role
`GET /api/vault`	Loads the current user’s saved credentials.
`POST /api/vault`	Creates a new encrypted password entry.
`PUT /api/vault/{id}`	Updates an existing entry.
`DELETE /api/vault/{id}`	Deletes an entry owned by the user.
`GET /api/vault/shared`	Loads credentials shared with the current user.

Passwords are encrypted before saving and decrypted only when the authorized user requests the data.

7. Sharing Workflow

1. User selects one of their own saved credentials.
2. User enters recipient email and permission level.
3. Backend normalizes the email and finds the recipient user case-insensitively.
4. Backend saves a share record in `shared_password_entries`.
5. Recipient logs in and opens the Shared Vault tab.
6. Backend finds rows for the recipient by user id or email, decrypts the password, and sends it to the UI.
7. UI shows the password with Show/Hide and Copy actions.

8. Password Reset Workflow

1. User enters registered email in the forgot-password flow.
2. Backend generates an OTP and sends it by email.
3. User enters the OTP to verify ownership.
4. User submits a new password.
5. Backend validates strength, hashes the password, updates the user, and consumes the reset token.

9. Database Model

- `users` : account data and verification state.
- `password_entries` : encrypted user credentials.
- `shared_password_entries` : share mapping between owner and recipient.
- `refresh_tokens` : session persistence for login refresh.
- `verification_tokens` : email verification tokens.
- `password_reset_tokens` : OTP records for password reset.

10. Important Code Paths

- `AuthServiceImpl` : registration, login, refresh token, logout, password reset.
- `PasswordVaultServiceImpl` : create, update, delete, share, decrypt, and load entries.
- `UserDetailsServiceImpl` : loads users for security and login.
- `JwtAuthenticationFilter` : reads JWT and authenticates requests.
- `Dashboard.jsx` : loads vault data and renders show/copy/share actions.

11. Interview Summary

When explaining the project, describe it as a secure credential manager with authenticated API access, encrypted password storage, email-based account verification, refresh-token login, controlled sharing, and password reset support. The key engineering points are authorization, encryption, and consistent state between frontend and backend.